Lecture 7~12 보충: 직관적 이해를 위한 심화 해설

> Diffusion, CLIP, Latent Diffusion, Flow Matching, DiT 등 후반부 강의의 핵심 개념들을 직관적으로 설명합니다.

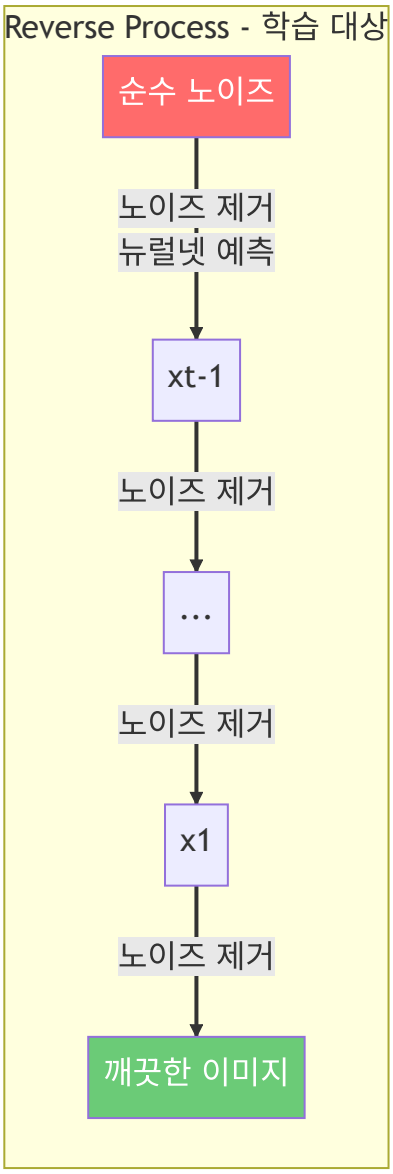
Lecture 7: DDPM (Denoising Diffusion Probabilistic Models)

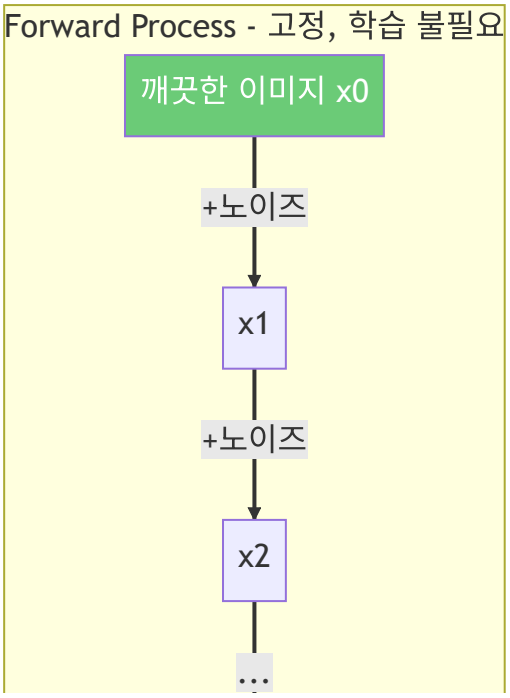
7.1 Diffusion의 핵심 아이디어 — 왜 "노이즈를 추가했다 제거"하는가?

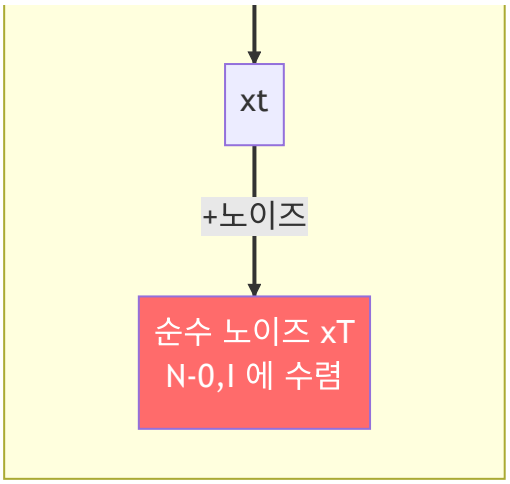
이것은 생성 모델 역사에서 가장 반직관적인 발상 중 하나다.

문제: 복잡한 데이터 분포 $p(x)$에서 직접 샘플링하는 것은 어렵다.

발상: "데이터에 점진적으로 노이즈를 추가하면 결국 순수 가우시안 노이즈가 된다. 이 과정을 **역으로** 배우면, 노이즈에서 데이터를 만들 수 있다!"


Reverse Process - 학습 대상
순수 노이즈
노이즈 제거
뉴럴넷 예측
xt-1
노이즈 제거
...
노이즈 제거
x1
노이즈 제거
깨끗한 이미지


Forward Process - 고정, 학습 불필요
깨끗한 이미지 x0
+노이즈
x1
+노이즈
x2
...



비유: 잉크 한 방울을 물에 떨어뜨리면 점차 퍼져서 균일해진다 (forward). 이 과정의 영상을 거꾸로 재생하면, 균일한 물이 점차 잉크 방울로 모인다 (reverse). Diffusion 모델은 이 "역재생"을 학습하는 것이다.

7.2 Forward Process — 왜 이 특정 노이즈 스케줄을 쓰는가?

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t} \cdot x_{t-1}, \beta_t I)$$

왜 $\sqrt{1-\beta_t}$와 β_t인가? 이것은 **분산이 보존되도록** 설계한 것이다.

x_{t-1}의 분산이 1이라면:

- $\sqrt{1-\beta_t} \cdot x_{t-1}$의 분산: $(1-\beta_t)$
- 추가 노이즈의 분산: β_t
- 합계: $(1-\beta_t) + \beta_t = 1$

분산을 1로 유지하는 이유: 분산이 스텝마다 커지면 값이 발산하고, 작아지면 정보가 사라진다. 1로 유지해야 안정적인 학습이 가능하다.

7.3 $\bar{\alpha}_t$의 등장 — 왜 한 번에 t 스텝 건너뛸 수 있는가?

핵심 공식:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} \cdot x_0, (1-\bar{\alpha}_t)I)$$

여기서 $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, $\alpha_t = 1 - \beta_t$

왜 이것이 중요한가? 학습 시 매번 t번의 노이즈 추가를 반복하지 않고, **한 번에** x_0에서 x_t를 샘플링할 수 있다:

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1-\bar{\alpha}_t} \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

유도 과정의 직관: 가우시안 노이즈를 더하는 것을 반복하면, 결과도 가우시안이다 (가우시안의 합은 가우시안). 따라서 t번 반복한 결과를 하나의 가우시안으로 합칠 수 있다.

$\bar{\alpha}_t$가 감소하면서:

- $t = 0$: $\bar{\alpha}_0 = 1$ → x_0 자체 (노이즈 없음)
- $t = T$: $\bar{\alpha}_T \approx 0$ → 순수 노이즈

7.4 왜 "노이즈를 예측"하는가? — ϵ-prediction의 직관

DDPM은 뉴럴넷 $\epsilon_\theta(x_t, t)$에게 **추가된 노이즈 ϵ을 예측**하라고 학습시킨다:

$$\mathcal{L} = \mathbb{E}_{t,x_0,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right]$$

왜 x_0를 직접 예측하지 않는가? 사실 수학적으로는 동치다:

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1-\bar{\alpha}_t} \cdot \epsilon)$$

ϵ을 알면 x_0를 복원할 수 있고, x_0를 알면 ϵ을 역산할 수 있다. 그런데 실험적으로 **ϵ을 예측하는 것이 더 안정적으로 학습**된다. 이유는:

1. $\epsilon \sim \mathcal{N}(0, I)$는 모든 t에서 같은 스케일 (분산 1)
2. x_0는 데이터에 따라 스케일이 다를 수 있음
3. 노이즈 예측은 "denoising"이라는 직관적 해석이 가능

7.5 DDPM과 Score-Based의 연결 — 같은 것의 다른 표현

$$\epsilon_\theta(x_t, t) \approx -\sqrt{1-\bar{\alpha}_t} \cdot \nabla_{x_t} \log p(x_t)$$

노이즈 예측 = Score의 스케일 변환이다!

- Score: "데이터 밀도가 높은 방향을 가리키는 화살표"
- 노이즈 예측: "추가된 노이즈의 방향" (이것을 빼면 데이터로 돌아감)
- 두 방향은 본질적으로 **같은 방향**을 가리킨다

이 연결이 중요한 이유: DDPM (Ho et al., 2020)과 Score-Based (Song & Ermon, 2019)이 독립적으로 발전했지만, Song et al., 2021 (Score SDE)에서 통합되었다. 둘 다 같은 것을 다른 관점에서 바라본 것이다.

7.6 Reverse Process의 수식 — 왜 이렇게 생겼는가?

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$$

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right)$$

이 복잡한 수식의 직관:

1. x_t에서 시작
2. 예측된 노이즈 ϵ_θ를 $\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}$ 비율로 빼줌

3. $\frac{1}{\sqrt{\alpha_t}}$로 스케일 보정

왜 한 번에 노이즈를 다 빼지 않는가? T스텝에 걸쳐 **조금씩** 제거하는 것이 핵심이다. 이유:

- 한 번에 제거하면 뉴럴넷이 너무 큰 변환을 한 번에 학습해야 함 → 어렵다
- 조금씩 제거하면 각 스텝의 변환이 작고 가우시안에 가까움 → 학습 용이
- 이것은 "큰 점프 한 번" vs "작은 걸음 천 번"의 차이

7.7 DDPM의 손실 함수와 ELBO의 연결

DDPM의 간단한 손실:

$$\mathcal{L}_{\text{simple}} = \|\epsilon - \epsilon_\theta(x_t, t)\|^2$$

이것이 VAE의 ELBO를 유도하는 과정에서 나온다:

$$\log p(x) \geq \text{ELBO} = -\sum_{t=1}^{T} \underbrace{D_{KL}(q(x_{t-1}|x_t, x_0) \| p_\theta(x_{t-1}|x_t))}_{\text{각 스텝의 denoising 품질}}$$

각 KL divergence 항이 가우시안 사이의 거리이므로, 결국 평균의 차이 = 노이즈 예측의 차이로 환원된다.

Variational Bound와의 관계: DDPM은 사실 T**스텝의 계층적 VAE**로 해석할 수 있다. x_T가 prior, 각 $x_{t-1}|x_t$가 하나의 디코더 층에 해당한다.

7.8 U-Net — 왜 Diffusion에서 이 아키텍처를 쓰는가?

DDPM의 노이즈 예측 네트워크로 U-Net을 사용한다. 왜?

U-Net의 구조:

```
입력 xₜ → [Down↓] → [Down↓] → [Down↓] → [Bottleneck] → [Up↑] → [Up↑]
                  │
                  └──────────── Skip Connection ──────────────────────
```

왜 U-Net인가?

1. **입출력 크기가 같다**: 입력 x_t와 출력 ϵ이 같은 크기
2. **다중 스케일 정보**: 다운샘플링으로 전역 구조를 파악하고, 스킵 연결로 세부 디테일을 보존
3. **노이즈 제거에 적합**: 원래 의료 영상 세그멘테이션용으로 설계되었는데, "입력과 비슷하지만 다른 것을 출력"해야 하는 구조가 denoising과 동일

Time Embedding의 역할: U-Net에 현재 타임스텝 t를 알려주는 것. Sinusoidal embedding으로 t를 벡터로 변환하여 각 레이어에 주입한다. t에 따라 노이즈 수준이

다르므로, 모델이 "지금 노이즈가 얼마나 심한지"를 알아야 적절한 제거를 할 수 있다.

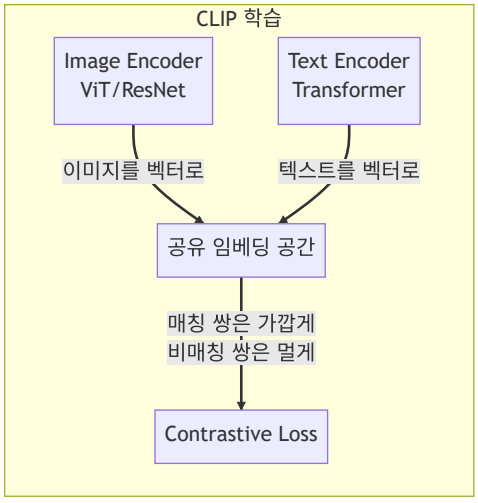
Lecture 8: CLIP & Classifier-Free Guidance

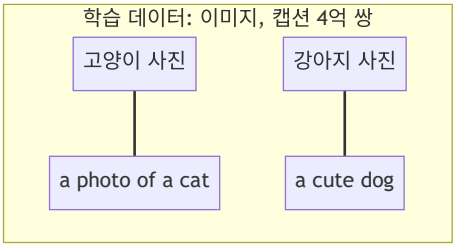
8.1 CLIP — 왜 이미지와 텍스트를 같은 공간에 넣는가?

$$\text{similarity}(I, T) = \frac{\text{Image_Embedding} \cdot \text{Text_Embedding}}{|\text{Image_Embedding}| \cdot |\text{Text_Embedding}|}$$

핵심 동기: 이미지 생성을 텍스트로 제어하려면, "고양이"라는 단어와 고양이 사진이 **같은 공간에서 가까이** 있어야 한다.

CLIP의 학습:





Contrastive Learning:

- 배치 내 N개의 (이미지, 텍스트) 쌍이 있을 때
- 올바른 쌍 (N개): 유사도를 높임
- 잘못된 쌍 ($N^2 - N$개): 유사도를 낮춤

왜 이것이 혁명적인가?

1. **Zero-shot 분류**: 새로운 카테고리를 학습 없이 인식 ("이 이미지가 'dog'에 가까운가 'cat'에 가까운가?")
2. **텍스트 기반 이미지 생성의 기반**: DALL-E 2, Stable Diffusion 등에서 조건부 생성에 사용
3. **4억 쌍**이라는 거대한 웹 데이터로 학습하여 매우 범용적

8.2 Classifier Guidance — 왜 분류기를 이용하면 품질이 올라가는가?

$$\nabla_{x_t} \log p(x_t|y) = \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(y|x_t)$$

이것은 **Bayes' rule의 기울기 버전**이다:

$$p(x|y) \propto p(x) \cdot p(y|x) \implies \log p(x|y) = \log p(x) + \log p(y|x) + \text{const}$$

양변에 ∇_x를 취하면 위 등식이 나온다.

직관:

- $\nabla_{x_t} \log p(x_t)$: "데이터 분포의 높은 곳으로 향하라" (무조건부 score)
- $\nabla_{x_t} \log p(y|x_t)$: "클래스 y일 확률이 높아지는 방향으로 향하라"
- 합: "데이터처럼 보이면서 동시에 클래스 y에 해당하는 방향으로 향하라"

Guidance Scale w:

$$\tilde{\nabla} = \nabla \log p(x_t) + w \cdot \nabla \log p(y|x_t)$$

$w > 1$이면 분류기 방향을 과도하게 강조한다. **품질-다양성 트레이드오프**:

- w 작으면: 다양하지만 조건에 부정확
- w 크면: 조건에 정확하지만 다양성 감소, 과포화

8.3 Classifier-Free Guidance (CFG) — 왜 분류기 없이도 되는가?

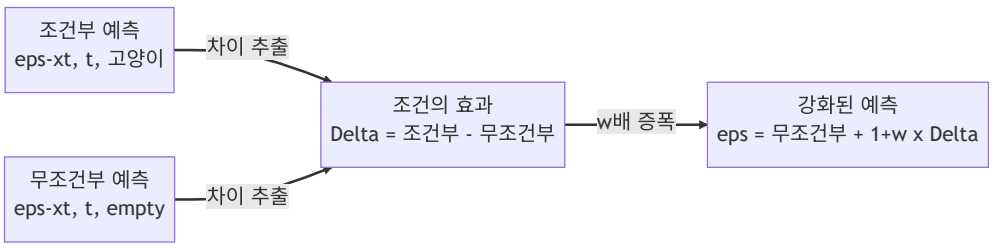
Classifier Guidance의 문제: 별도의 노이즈 이미지용 분류기를 훈련해야 한다.

CFG의 핵심 트릭:

$$\tilde{\epsilon}_\theta(x_t, t, y) = (1 + w) \cdot \epsilon_\theta(x_t, t, y) - w \cdot \epsilon_\theta(x_t, t, \varnothing)$$

- $\epsilon_\theta(x_t, t, y)$: 조건부 예측 ("고양이를 생각하며 노이즈 제거")
- $\epsilon_\theta(x_t, t, \varnothing)$: 무조건부 예측 ("아무 생각 없이 노이즈 제거")
- 차이: 조건이 미치는 영향
- $(1 + w)$ 배로 강조: 조건의 영향을 증폭

직관: "조건 있을 때와 없을 때의 차이 = 조건의 효과"를 추출하여 증폭시키는 것이다.



훈련 트릭: 학습 시 일정 확률(보통 10%)로 조건 y를 빈 값($\varnothing$)으로 대체한다. 이렇게 하면 하나의 모델이 조건부/무조건부 양쪽을 모두 학습한다.

왜 CFG가 표준이 되었는가?

1. 별도 분류기 불필요 → 구현 간단
2. 하나의 모델로 조건부/무조건부 통합
3. 생성 품질이 Classifier Guidance와 동등 이상
4. guidance scale w로 품질-다양성 조절 용이

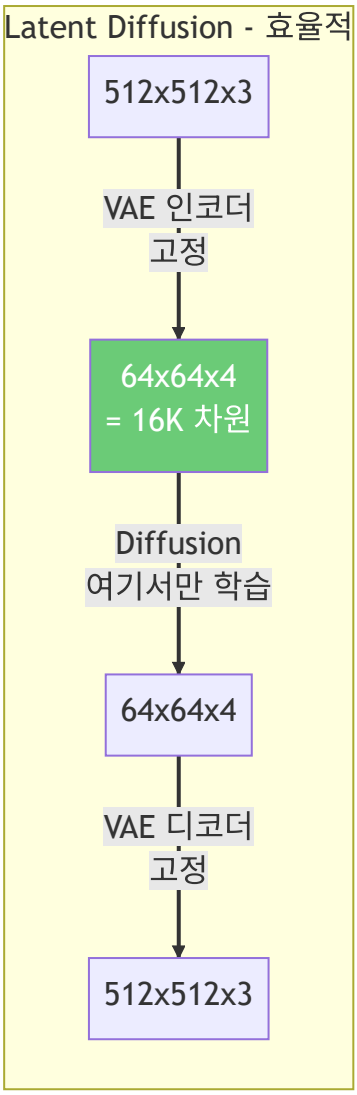
Lecture 9: Latent Diffusion & Stable Diffusion

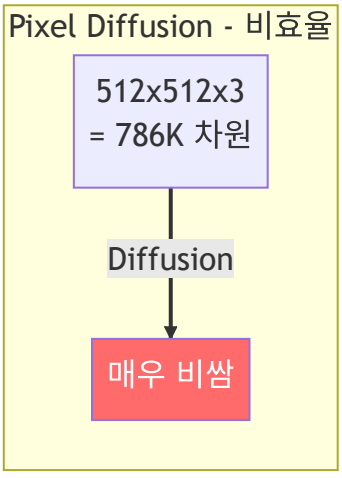
9.1 왜 "Latent" 공간에서 Diffusion을 하는가?

픽셀 공간 Diffusion의 문제:

- 512×512×3 이미지: 786,432차원에서 Diffusion → 계산량 폭발
- U-Net이 고해상도에서 동작해야 함 → 메모리 부족

해결: 이미지를 먼저 **저차원 잠재 공간으로 압축**한 뒤, 거기서 Diffusion을 수행





압축률: $\frac{786432}{16384} = 48$배 차원 축소. 계산량은 차원의 제곱에 비례하므로, 실질적으로 **~2000배** 빨라진다.

왜 정보 손실이 적은가? VAE 인코더가 이미지의 "본질적 정보"만 잠재 공간에 보존하고, "인지적으로 무의미한 세부사항"(고주파 노이즈 등)은 버린다. KL regularization이 잠재 공간을 매끄럽게 만들어, Diffusion이 동작하기 좋은 공간이 된다.

9.2 조건부 생성의 구조 — Cross-Attention의 역할

Stable Diffusion에서 텍스트 조건은 **Cross-Attention**으로 주입된다:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

여기서:

- $Q = W_Q \cdot \phi_{\text{image}}(x_t)$ (이미지 피처에서 유도)
- $K = W_K \cdot \tau_\theta(y)$ (텍스트 임베딩에서 유도)
- $V = W_V \cdot \tau_\theta(y)$ (텍스트 임베딩에서 유도)

직관: U-Net의 각 레이어에서 이미지 피처가 "이 부분을 생성할 때 텍스트의 어떤 단어를 참고해야 할까?"라고 물어본다. 고양이의 귀를 생성하는 위치는 "cat"이라는 단어에 높은 attention을 가지고, 하늘을 생성하는 위치는 "sky"에 높은 attention을 가진다.

9.3 CLIP Text Encoder — 왜 별도 텍스트 인코더가 필요한가?

텍스트 "a cat sitting on a couch"를 어떻게 모델에 전달하는가?

1. CLIP Text Encoder (또는 T5 등)가 텍스트를 **토큰 시퀀스의 임베딩**으로 변환
2. 이 임베딩이 Cross-Attention의 Key/Value가 됨

왜 직접 텍스트를 넣지 않는가? 뉴럴넷은 문자열을 직접 처리하지 못한다. 의미적으로 풍부한 **벡터 표현**으로 변환해야 한다. CLIP의 텍스트 인코더는 4억 쌍으로 학습되어, "cat"과 고양이 이미지가 가까운 벡터를 출력하도록 훈련되어 있다. 이 사전 학습된 의미 표현을 활용하는 것이 핵심이다.

9.4 Noise Schedule과 SNR — 왜 특정 스케줄이 중요한가?

SNR(Signal-to-Noise Ratio): $\text{SNR}(t) = \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t}$

- t 작을 때: SNR 높음 → 신호(이미지) 강함 → 세부 디테일 학습
- t 클 때: SNR 낮음 → 노이즈 강함 → 전체 구조 학습

직관: 사진이 매우 흐릿할 때(큰 t) "이건 사람이네" 정도만 알 수 있다 (전체 구조). 선명해질수록(작은 t) "눈이 갈색이네" 같은 세부사항을 파악할 수 있다. 모델은 t에 따라 다른 수준의 정보를 학습한다.

Lecture 10: Flow Matching

10.1 Flow Matching — 왜 Diffusion의 대안이 필요한가?

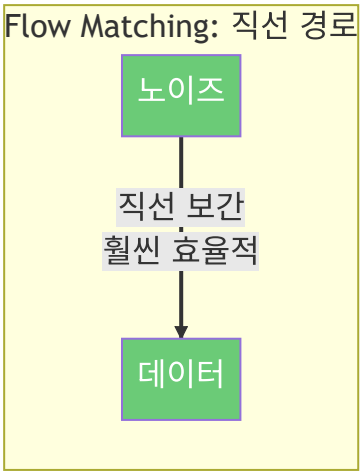
Diffusion의 단점:

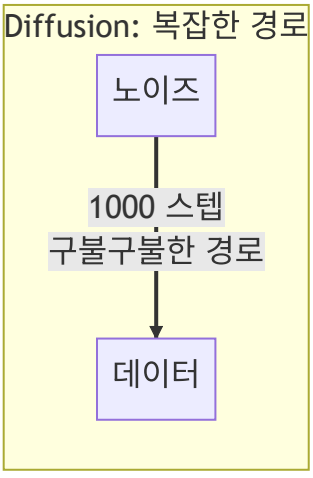
1. 생성 시 1000스텝 반복 → 느림
2. 이산적 타임스텝 → 수학적으로 지저분
3. 노이즈 스케줄 선택이 중요하고 민감

Flow Matching의 핵심 아이디어: 노이즈 $x_0 \sim \mathcal{N}(0, I)$에서 데이터 $x_1 \sim p_{\text{data}}$까지의 **직선 경로**를 학습한다.

$$x_t = (1-t) \cdot x_0 + t \cdot x_1, \quad t \in [0, 1]$$

직관: Diffusion은 "천 계단을 오르내리는" 방식이라면, Flow Matching은 "출발점에서 도착점까지 직선으로 이동하는" 방식이다.





10.2 Velocity Field — 무엇을 학습하는가?

$$v_\theta(x_t, t) \approx \frac{dx_t}{dt} = x_1 - x_0$$

모델이 학습하는 것은 **속도장(velocity field)** — "현재 위치 x_t와 시간 t에서, 어떤 방향으로 얼마나 이동해야 하는가?"

Diffusion과의 비교:

- Diffusion: "x_t에 추가된 노이즈 ϵ을 예측하라"
- Flow Matching: "x_t의 이동 방향 v를 예측하라"

손실 함수:

$$\mathcal{L} = \mathbb{E}_{t,x_0,x_1}\left[\|v_\theta(x_t,t) - (x_1 - x_0)\|^2\right]$$

직선 보간에서 속도는 상수 $v = x_1 - x_0$이므로, 매우 깔끔하다.

10.3 ODE vs SDE — 왜 Flow Matching은 ODE를 쓰는가?

SDE (Stochastic Differential Equation): Diffusion 모델에서 사용. 각 스텝에 노이즈가 추가됨. → 확률적, 매 실행마다 다른 결과.

ODE (Ordinary Differential Equation): Flow Matching에서 사용. 결정적 경로. → 같은 시작점에서 항상 같은 결과.

$$\frac{dx_t}{dt} = v_\theta(x_t,t) \quad (\text{ODE — 결정적})$$

ODE의 장점:

1. 적응적 스텝 크기 사용 가능 → 적은 스텝으로 빠른 생성
2. 역방향 계산 가능 → 이미지 → 잠재 벡터 역변환 (inversion)
3. 정확한 log-likelihood 계산 가능

10.4 Conditional Flow Matching — 왜 "조건부"가 필요한가?

전체 velocity field $v(x_t,t)$를 직접 학습하는 것은 어렵다. 대신 **개별 데이터 쌍 (x_0,x_1)에 대한 조건부 flow**를 학습한 뒤, 이를 합쳐서 전체 flow를 근사한다.

이것이 작동하는 이유: 개별 직선 경로들의 평균이 전체 분포를 연결하는 flow가 된다. 각 "입자"가 자기만의 직선 경로를 따라 이동하면, 전체 "입자 구름"의 분포가 가우시안에서 데이터 분포로 변환된다.

10.5 Rectified Flow — "직선화"란 무엇인가?

기본 Flow Matching에서 개별 경로는 직선이지만, 전체 ODE를 풀면 경로가 약간 구부러질 수 있다 (다른 입자들과의 상호작용 때문). **Rectified Flow**는 이 경로를 더 직선에 가깝게 만드는 기법이다.

직선에 가까울수록 → 적은 ODE 스텝으로 정확한 생성 → **더 빠르다**.

Stable Diffusion 3와 Flux가 이 기법을 사용한다.

Lecture 11: DiT, Consistency Models, VAR

11.1 DiT (Diffusion Transformer) — 왜 U-Net을 Transformer로 교체하는가?

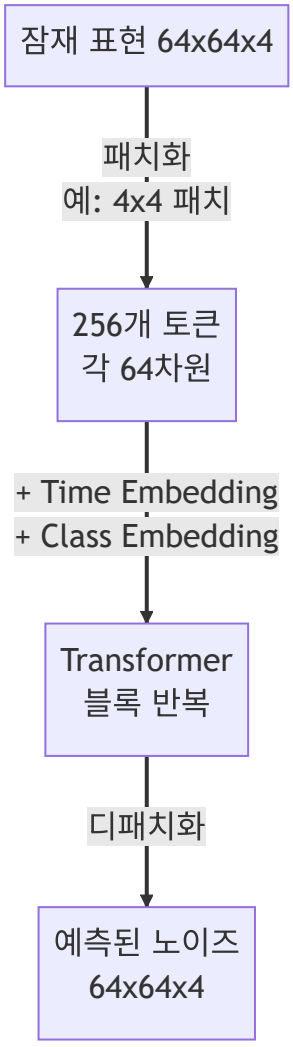
U-Net의 한계:

1. 해상도별 다른 구조 → 확장(scaling)이 어려움
2. 아키텍처 설계에 많은 수작업 필요
3. CNN 기반이라 장거리 의존성에 약함

Transformer의 장점:

1. **Scaling Law**: 모델을 키우면 성능이 예측 가능하게 향상 (GPT에서 검증됨)
2. 균일한 구조: 같은 블록을 반복하면 됨
3. Self-Attention으로 전역 정보 처리

DiT의 핵심: 잠재 공간의 피처맵을 **패치로 잘라서 토큰화**한 뒤, Transformer에 넣는다.



AdaLN (Adaptive Layer Normalization): DiT에서 시간 정보 t와 클래스 정보 y를 Transformer에 주입하는 방법. StyleGAN의 AdaIN과 같은 원리:

$$\mathrm{AdaLN}(h, y) = y_s \cdot \mathrm{LayerNorm}(h) + y_b$$

여기서 y_s, y_b는 시간/클래스 임베딩에서 생성된다.

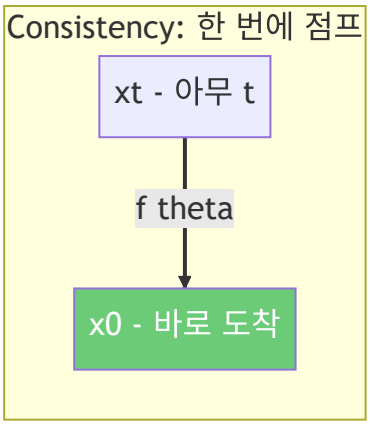
11.2 Consistency Models — 왜 한 스텝으로 생성하고 싶은가?

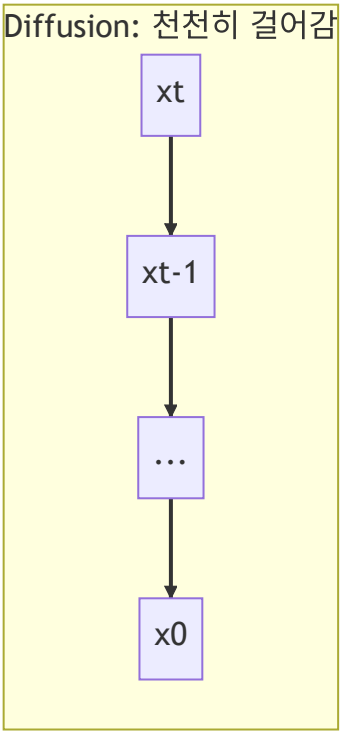
문제: Diffusion은 수십~수천 스텝이 필요하다. 실시간 응용(비디오 생성, 인터랙티브 편집)에서 너무 느리다.

핵심 아이디어: ODE 궤적 위의 모든 점이 **같은 시작점(데이터)에 매핑**되는 함수를 학습한다.

$$f_\theta(x_t, t) = x_0 \quad \text{for all } t$$

즉, 어떤 시점 t의 노이즈 이미지를 넣어도, 한 번에 깨끗한 이미지 x_0으로 점프한다.





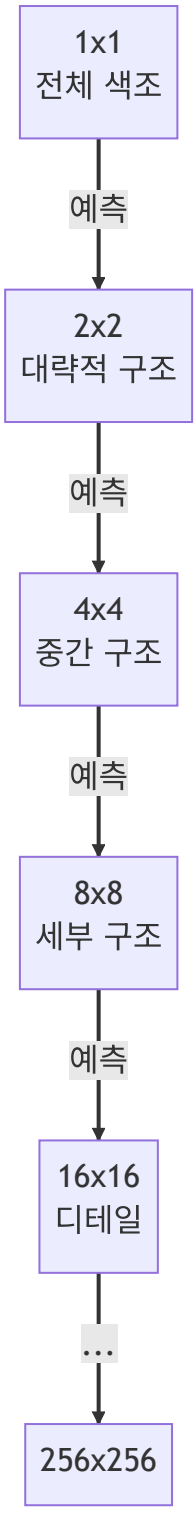
Consistency 조건: 같은 ODE 궤적 위의 모든 점에 대해 f_θ의 출력이 동일해야 한다:

$$f_\theta(x_t, t) = f_\theta(x_{t'}, t') \quad \text{같은 궤적 위의 } t, t'$$

11.3 VAR (Visual Autoregressive Model) — 왜 "다음 스케일"을 예측하는가?

기존 자기회귀 모델: 다음 토큰(패치)을 예측 → 래스터 순서로 좌→우, 상→하

VAR의 혁신: 다음 **해상도 스케일**을 예측



왜 이것이 자연스러운가? 화가가 그림을 그리는 과정을 생각하면:

1. 먼저 전체 구도를 대략적으로 스케치 (저해상도)
2. 점차 세부사항을 추가 (고해상도)

이것은 래스터 순서(좌→우, 상→하)보다 **훨씬 자연스럽다**. 인간도 그림을 왼쪽 위 픽셀부터 그리지 않는다.

장점:

- coarse-to-fine 생성으로 전체 구조가 먼저 결정됨
- 각 스케일 내에서는 병렬 처리 가능 → Diffusion보다 빠름
- GPT와 같은 자기회귀 구조 + Diffusion의 다중 스케일 장점

Lecture 12: Video, 3D, Physical AI

12.1 비디오 생성 — 왜 이미지보다 훨씬 어려운가?

$$\text{비디오} = \text{이미지 시퀀스} + \text{시간적 일관성}(\text{temporal consistency})$$

512×512 이미지는 786,432차원. 30fps 5초 비디오는 $786432 \times 150 = 1.18$억 차원. 차원이 **150배** 증가한다.

그런데 차원 문제보다 더 근본적인 문제는 **시간적 일관성**이다:

- 프레임마다 독립적으로 이미지를 생성하면 깜빡거림(flickering)
- 물체의 움직임이 물리적으로 타당해야 함
- 카메라 시점 변화가 자연스러워야 함

12.2 3D Diffusion — NeRF와의 연결

NeRF (Neural Radiance Field): 여러 각도의 2D 사진으로부터 3D 장면을 학습하는 모델

$$F_\theta : (x, y, z, \theta, \phi) \to (r, g, b, \sigma)$$

3D 좌표 + 시선 방향 → 색상 + 밀도

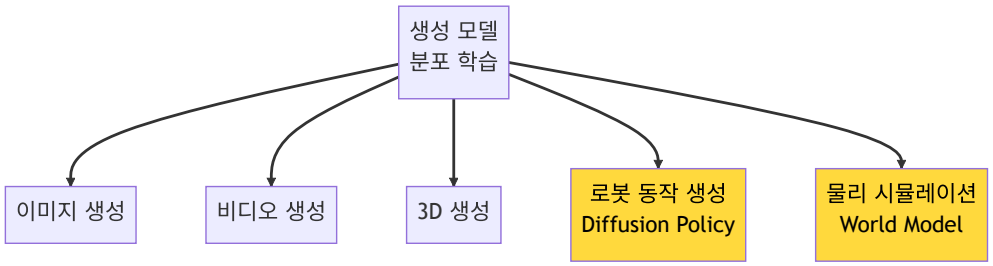
Score Distillation Sampling (SDS): 텍스트 → 3D 생성에서 핵심 기법

사전 학습된 2D Diffusion 모델의 지식을 활용하여 3D 모델을 최적화한다. 다양한 시점에서 렌더링한 이미지가 Diffusion 모델이 생성할 법한 이미지가 되도록 NeRF를 업데이트한다.

12.3 Physical AI — 왜 물리 시뮬레이션과 생성 모델이 만나는가?

핵심 연결: 생성 모델은 "데이터 분포를 학습"한다. 물리 세계의 데이터(로봇 동작, 유체 흐름 등)도 특정 분포를 따른다. 생성 모델로 이 분포를 학습하면:

1. **로봇 동작 생성**: "컵을 잡아"라는 명령 → 동작 시퀀스 생성 (Diffusion Policy)
2. **물리 시뮬레이션 가속**: 수치적 시뮬레이션 대신 학습된 모델로 빠르게 예측
3. **World Model**: 환경의 동역학을 학습하여 미래를 예측



12.4 Diffusion Policy — 왜 로봇 제어에 Diffusion을 쓰는가?

전통적 로봇 제어: 상태 → 하나의 최적 행동 (결정적 정책)

문제: "컵을 잡는 방법"은 여러 가지가 있다 (위에서, 옆에서, 손가락 배치 등). **다봉 분포(multimodal distribution)**다.

Diffusion Policy: 상태 → **행동의 분포를 생성** → 다양한 유효 행동 중 하나를 샘플링

이것은 GAN의 mode collapse 문제를 Diffusion이 해결한 것과 같은 원리다. Diffusion은 다봉 분포를 잘 모델링한다.

전체 강의를 관통하는 핵심 패턴들

패턴 1: "어려운 문제를 쉬운 문제의 연쇄로 분해"

모델	어려운 문제	쉬운 문제로 분해
PixelRNN	$p(x)$ 전체 모델링	조건부 확률 $p(x_i \| x_{<i})$의 곱
VAE	$p(x)$ 직접 최대화	ELBO (하한)를 최대화
Diffusion	노이즈→데이터 한 번에	1000개의 작은 denoising 스텝
Flow Matching	복잡한 변환	직선 보간

패턴 2: "학습 가능한 것과 고정된 것의 분리"

모델	고정 (학습 불필요)	학습 대상
VAE	Prior $p(z) = \mathcal{N}(0, I)$	인코더 q_ϕ, 디코더 p_θ
Diffusion	Forward process (노이즈 추가)	Reverse process (노이즈 제거)
Latent Diffusion	VAE 인코더/디코더	U-Net/DiT
VQ-VAE	—	인코더, 디코더, 코드북 모두

패턴 3: "압축 → 생성"의 2단계 패턴

모델	Stage 1 (압축)	Stage 2 (생성)
VQ-VAE	이미지 → 코드 인덱스	PixelCNN으로 코드 생성
VQ-GAN	이미지 → 코드 인덱스 (GAN으로 향상)	Transformer로 코드 생성
Latent Diffusion	이미지 → 잠재 벡터 (VAE)	Diffusion으로 잠재 벡터 생성
DALL-E	이미지 → dVAE 토큰	Transformer로 토큰 생성

이 패턴이 반복되는 이유: **고차원 데이터에서 직접 생성하는 것은 비효율적**이므로, 먼저 저차원으로 압축하고 거기서 생성하는 것이 항상 더 효율적이다.

보충: 빠진 핵심 주제들

보충 7.9 DDIM — 왜 1000스텝을 50스텝으로 줄일 수 있는가?

DDPM의 문제: 생성 시 반드시 $T = 1000$ 스텝을 **순차적으로** 모두 거쳐야 한다. 한 스텝에 GPU 연산 한 번이 필요하므로, 512×512 이미지 하나 생성에 수십 초가 걸린다. 실시간 응용은 불가능하다.

DDIM의 핵심 통찰: 역과정(reverse process)이 반드시 **확률적(stochastic)**일 필요가 없다!

DDPM과 DDIM의 차이를 정확히 이해하자:

- **DDPM**: 각 역방향 스텝에서 예측된 노이즈를 빼고, **새로운 랜덤 노이즈를 다시 추가**한다 (확률적).
- **DDIM**: 예측된 노이즈를 빼기만 하고, **새 노이즈를 추가하지 않는다** (결정론적).

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \cdot \underbrace{\frac{x_t - \sqrt{1-\bar{\alpha}_t} \cdot \epsilon_\theta}{\sqrt{\bar{\alpha}_t}}}_{\text{예측된 } x_0} + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \epsilon_\theta + \underbrace{\sigma_t \cdot z}_{\text{DDPM: } \sigma_t > 0,\ \text{DDIM: } \sigma_t = 0}$$

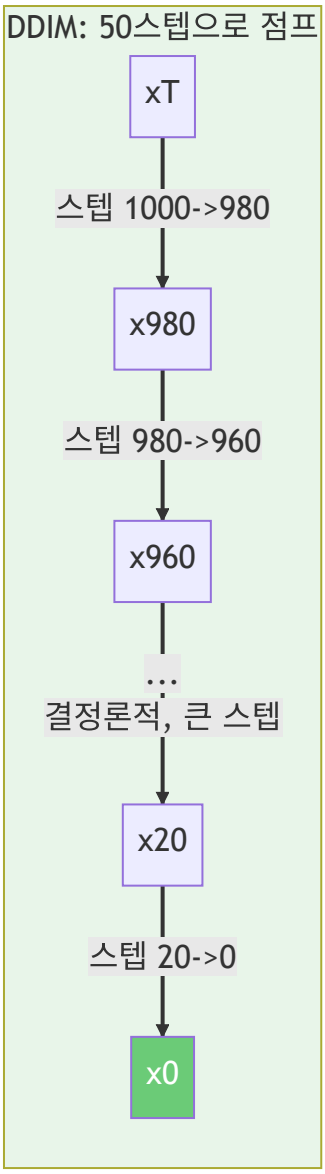
η **파라미터**: $\sigma_t = \eta \cdot \tilde{\sigma}_t$로 제어한다.

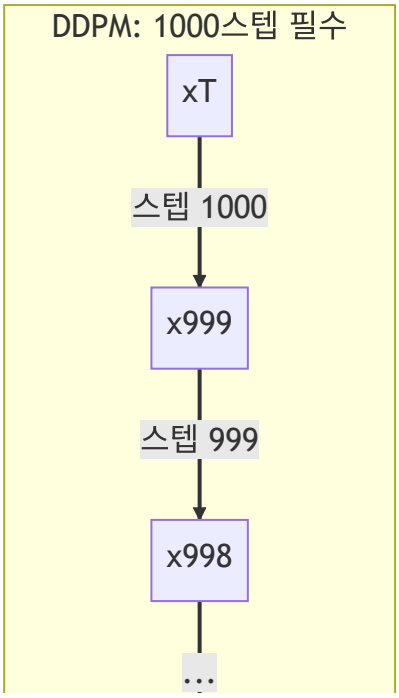
- $\eta = 0$: 완전 결정론적 (DDIM) — 같은 초기 노이즈 → 항상 같은 이미지
- $\eta = 1$: 완전 확률적 (DDPM) — 같은 초기 노이즈 → 매번 다른 이미지

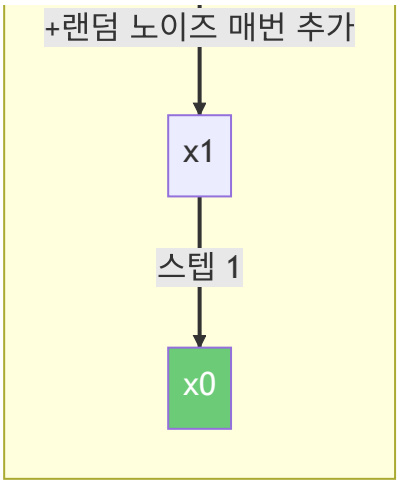
부분 시퀀스 트릭 (Subsequence Trick): $\eta = 0$이면 ODE가 되므로, 궤적이 **연속적**이다. 연속 궤적에서는 큰 스텝을 밟아도 된다!

$$t = 1000, 999, 998, \ldots, 1 \quad (1000\text{ 스텝}) \implies t = 1000, 980, 960, \ldots, 20 \quad (50\text{ 스텝})$$

이것이 작동하는 이유: DDIM은 본질적으로 **Probability Flow ODE의 오일러 방법 (Euler method)**이다. ODE를 수치적으로 풀 때 스텝 크기를 키울 수 있는 것과 같은 원리다.


DDIM: 50스텝으로 점프
xT
스텝 1000->980
x980
스텝 980->960
x960
...
결정론적, 큰 스텝
x20
스텝 20->0
x0


DDPM: 1000스텝 필수
xT
스텝 1000
x999
스텝 999
x998
...



비유: DDPM은 "1000개 계단을 한 칸씩 내려가면서 매 계단마다 동전을 던져 미세 조정하는 것"이고, DDIM은 "엘리베이터를 타고 20층씩 건너뛰며 내려가는 것"이다. 경로가 결정론적이므로 중간 층을 건너뛸 수 있다.

```
# DDIM 샘플링 (결정론적)
def ddim_sample(model, x_T, timesteps, alpha_bar):
    x_t = x_T
    for i in range(len(timesteps) - 1):
        t = timesteps[i]          # 현재 타임스텝
        t_prev = timesteps[i+1]   # 이전 타임스텝 (더 깨끗한 쪽)
        eps = model(x_t, t)       # 노이즈 예측
        # x0 예측
        x0_pred = (x_t - sqrt(1 - alpha_bar[t]) * eps) / sqrt(alpha_
        # 다음 스텝으로 점프
        x_t = sqrt(alpha_bar[t_prev]) * x0_pred + sqrt(1 - alpha_ba
    return x_t
```

보충 7.10 노이즈 스케줄 — Linear vs Cosine

노이즈 스케줄은 $\bar{\alpha}_t$가 t에 따라 어떻게 감소하는지를 결정한다. 이 선택이 생성 품질에 **크게** 영향을 미친다.

Linear Schedule (원래 DDPM):

- β_t가 0.0001에서 0.02까지 선형 증가
- **문제**: 초반에 노이즈가 너무 빠르게 추가되어 이미지 정보가 일찍 파괴된다
- $\bar{\alpha}_t$가 중간 영역에서 급격히 떨어짐 → 모델이 학습해야 할 "유용한" t 범위가 좁다

Cosine Schedule (Improved DDPM, Nichol & Dhariwal 2021):

- $\bar{\alpha}_t$가 코사인 곡선을 따라 **완만하게** 감소

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos\left(\frac{t/T + s}{1+s} \cdot \frac{\pi}{2}\right)^2$$

- 노이즈 추가가 더 **점진적** → 이미지 구조가 더 오래 보존 → 특히 작은 이미지에서 품질 향상

해상도에 따른 선택:

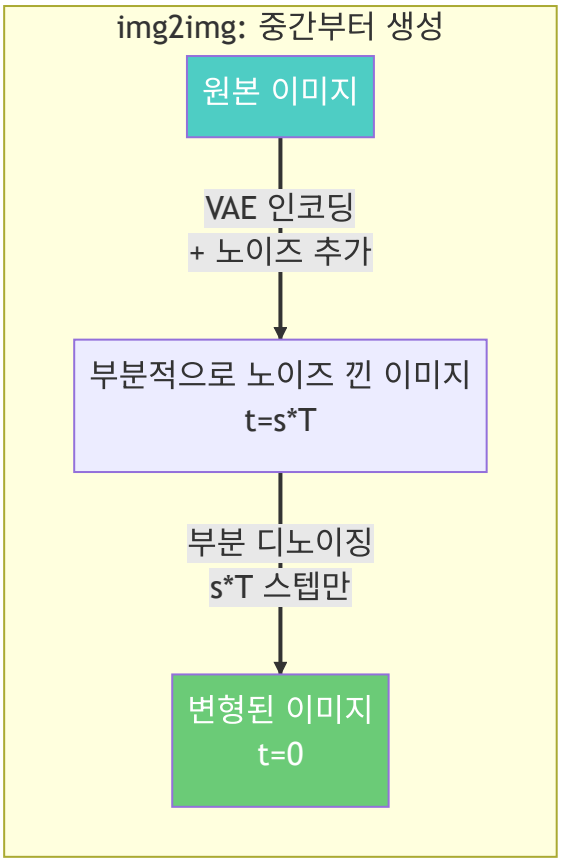
해상도	권장 스케줄	이유
32×32 ~ 64×64	Cosine	정보량이 적어서 linear로는 너무 빨리 파괴됨
256×256 이상	Linear / Scaled Linear	정보량이 많아서 linear도 충분히 점진적

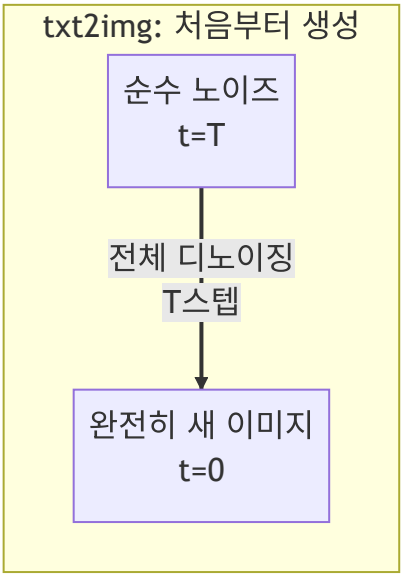
SNR과의 연결: 스케줄은 $\mathrm{SNR}(t) = \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t}$를 결정한다. Cosine 스케줄은 SNR이 더 균일하게 분포되어, 모든 노이즈 수준에서 고르게 학습이 이루어진다.

보충 9.5 Image-to-Image (img2img) — 왜 중간부터 시작하면 되는가?

txt2img는 순수 노이즈 x_T에서 시작하여 완전히 새로운 이미지를 생성한다. 그런데 "이미 있는 이미지를 기반으로 변형하고 싶다"면 어떻게 해야 하는가?

핵심 아이디어: 원본 이미지에 **중간 정도의 노이즈만 추가**하고, 거기서부터 디노이징을 시작한다.





strength 파라미터 $s \in [0, 1]$:

- $s = 0.0$: 노이즈를 전혀 추가하지 않음 → 원본과 거의 동일
- $s = 0.3$: 약간의 노이즈 → 색조, 질감 변경 정도
- $s = 0.7$: 상당한 노이즈 → 구도는 유지하되 내용이 크게 변함
- $s = 1.0$: 완전 노이즈 → txt2img와 동일 (원본 무시)

수식:

$$z_{\text{start}} = \sqrt{\bar{\alpha}_t} \cdot z_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon, \quad t = \lfloor s \cdot T \rfloor$$

비유: txt2img는 "백지에서 그림 그리기"이고, img2img는 "흐릿한 사진 위에 스케치하기"다. 사진이 더 흐릿할수록(strength가 높을수록) 원본에서 더 자유롭게 벗어난다.

```
# img2img 핵심 로직
def img2img(model, vae, image, prompt, strength=0.7):
    z_0 = vae.encode(image)           # 원본을 잠재 공간으로 인코딩
    t_start = int(strength * T)        # 시작 타임스텝 결정
    noise = torch.randn_like(z_0)      # 랜덤 노이즈 생성
    z_t = sqrt_alpha_bar[t_start] * z_0 + sqrt_one_minus_alpha_bar[t
    # t_start부터 0까지 디노이징 (전체가 아닌 일부만 수행)
    for t in range(t_start, 0, -1):
        z_t = denoise_step(model, z_t, t, prompt)
    return vae.decode(z_t)             # 잠재 공간에서 이미지로 복원
```

보충 9.6 Inpainting — 왜 마스크 영역만 생성할 수 있는가?

Inpainting은 이미지의 **특정 영역만 새로 생성**하고, 나머지는 원본을 유지하는 기법이다.

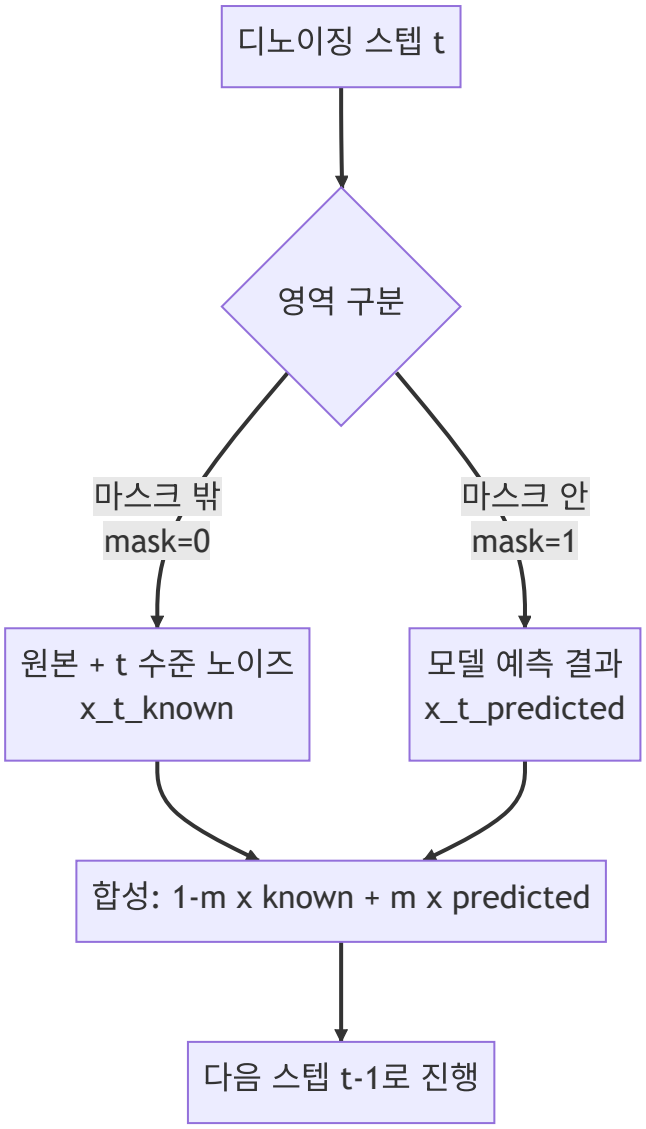
원리: 디노이징의 매 스텝에서 두 가지 소스를 **합성**한다:

$$x_t = (1 - m) \cdot x_t^{\text{known}} + m \cdot x_t^{\text{predicted}}$$

- **마스크 밖 (알려진 영역)**: 원본 이미지에 현재 시점 t에 맞는 노이즈를 추가한 값 x_t^{known}을 사용
- **마스크 안 (생성할 영역)**: 모델이 예측한 디노이징 결과 $x_t^{\text{predicted}}$를 사용

매 스텝마다 알려진 영역을 **강제로 원본 값으로 되돌리기** 때문에, 모델은 마스크 안쪽만 자유롭게 생성하면서도 경계가 자연스럽게 이어진다.

비유: "퍼즐을 맞추는 것"과 같다. 테두리 조각(알려진 영역)은 이미 놓여 있고, 가운데 빈 부분만 맞추면 된다. 주변 조각의 패턴을 참고하여 빈 곳을 채운다.

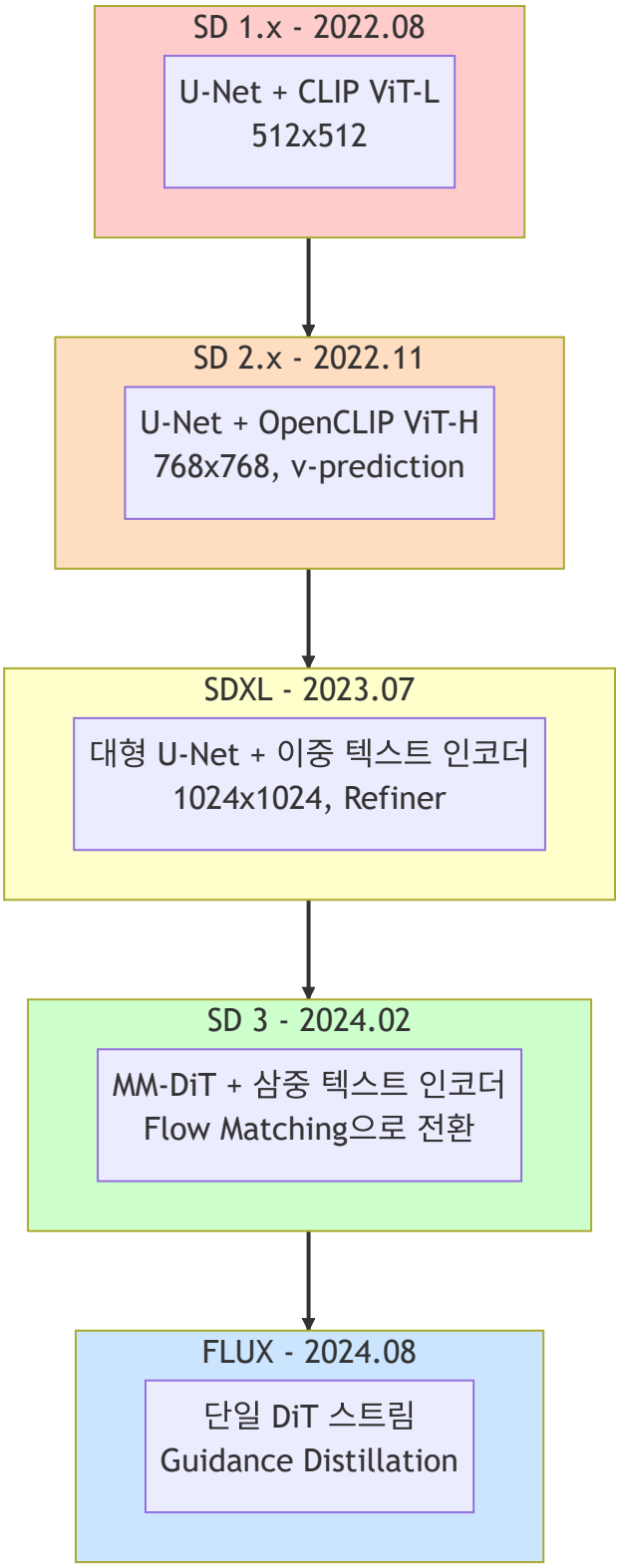


보충 9.7 Stable Diffusion 버전 진화 — SD 1.x에서 FLUX까지

Stable Diffusion은 오픈소스 이미지 생성 모델의 대표 주자다. 2022년부터 빠르게 진화해 왔다.

버전	출시	백본	텍스트 인코더	해상도	핵심 변화
SD 1.x	2022.08	U-Net	CLIP ViT-L	512×512	최초 오픈소스 LDM
SD 2.x	2022.11	U-Net	OpenCLIP ViT-H	768×768	v-prediction 도입
SDXL	2023.07	대형 U-Net	CLIP-G + CLIP-L (이중)	1024×1024	Refiner 모델, 이중 텍스트 인코더

SD 3	2024.02	MM-DiT	CLIP-L + CLIP-G + T5-XXL (삼중)	1024×1024	Flow Matching, Transformer로 전환
FLUX	2024.08	단일 DiT	CLIP-L + T5-XXL	1024×1024+	Guidance Distillation, 최고 품질



핵심 패러다임 전환 세 가지:

1. **U-Net** → **DiT** (CNN → Transformer): Scaling law의 혜택을 받기 위해. 모델을 키우면 예측 가능하게 성능이 향상된다.
2. **DDPM** → **Flow Matching** (SDE → ODE): 더 직선적인 경로로 적은 스텝에 고품질 생성이 가능하다.

3. **단일 텍스트 인코더 → 다중 텍스트 인코더**: CLIP은 시각-언어 정렬에 강하고, T5는 복잡한 텍스트 이해에 강하다. 두 장점을 결합한다.

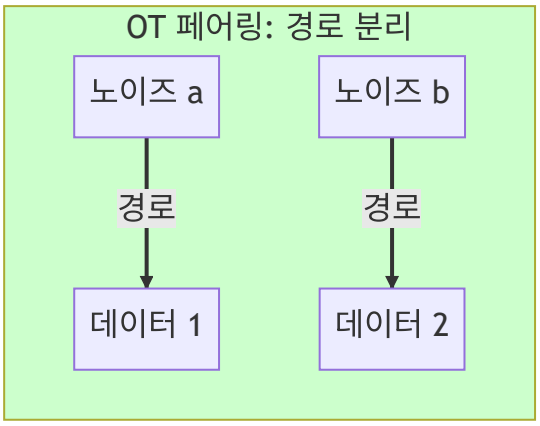
보충 10.5 OT-CFM — 왜 Optimal Transport가 더 나은 경로를 만드는가?

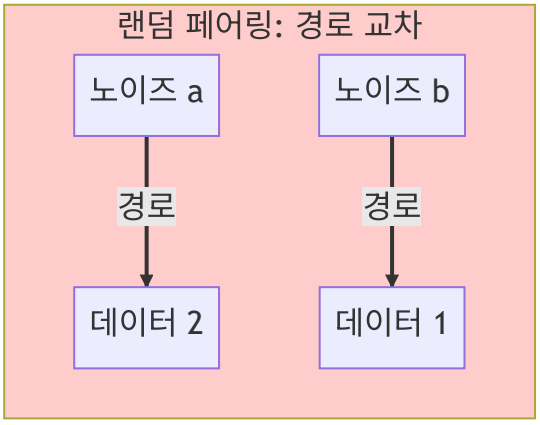
기본 Flow Matching에서는 노이즈 x_0와 데이터 x_1을 **랜덤하게 쌍을 맺는다**. 이 랜덤 페어링은 **경로가 서로 교차**하는 문제를 일으킨다.

경로 교차가 나쁜 이유: 속도장 $v(x_t, t)$이 같은 위치에서 서로 다른 방향을 가리킴 → 학습이 어렵고 ODE 해석이 필요한 스텝 수가 증가한다.

OT(Optimal Transport) 페어링: 미니배치 내에서 노이즈와 데이터 사이의 **최소 비용 매칭**을 찾는다.

$$\text{Cost} = \sum_i \|x_0^{(i)} - x_1^{\pi(i)}\|^2 \quad \text{를 최소화하는 순열 } \pi$$





직관: 랜덤 페어링은 "서울에 있는 사람이 부산 택시를 타고, 부산 사람이 서울 택시를 타는 것"이다. OT는 "각자 가장 가까운 택시를 타는 것" — 경로가 교차하지 않고 총 이동 거리가 최소화된다.

미니배치 OT: 전체 데이터에 대한 OT는 계산 비용이 크므로, **미니배치 내에서만** 할당 문제(assignment problem)를 풀어 근사한다. 이것만으로도 상당한 효과가 있다.

결과: 경로가 교차하지 않으면 → 속도장이 더 매끄러움 → 적은 ODE 스텝으로 정확한 생성이 가능하다.

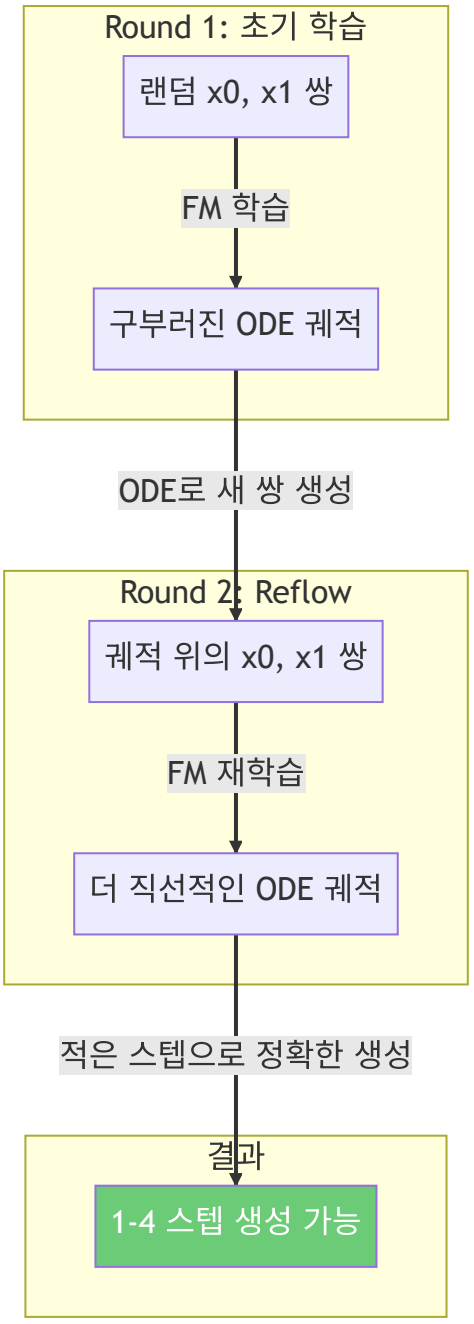
보충 10.6 Reflow — 경로를 "곧게 펴는" 방법

OT-CFM으로도 경로가 완벽히 직선은 아닐 수 있다. **Reflow**는 학습된 모델을 이용하여 경로를 **반복적으로 직선화**하는 기법이다.

알고리즘:

1. **1단계**: 일반적인 Flow Matching 모델을 학습한다.
2. **2단계**: 학습된 모델로 (x_0, x_1) 쌍을 생성한다 — $x_0 \sim \mathcal{N}(0, I)$에서 ODE를 풀어 x_1을 얻는다.
3. **3단계**: 이 새로운 (x_0, x_1) 쌍으로 Flow Matching 모델을 **재학습**한다.
4. (선택적) 2-3단계를 반복한다 (Reflow 2, Reflow 3...)

왜 이것이 작동하는가? 2단계에서 생성한 (x_0, x_1) 쌍은 이미 **학습된 ODE 궤적 위에** 있다. 이 쌍으로 다시 직선 보간을 학습하면, ODE 궤적 자체가 직선에 더 가까워진다.



실용적 의의: Stable Diffusion 3는 1라운드의 Reflow(Rectified Flow)를 사용하여, 기존 Diffusion 대비 훨씬 적은 스텝으로 고품질 이미지를 생성한다.

보충 10.7 고차 ODE 솔버 — Euler vs Heun vs RK4

Flow Matching이나 DDIM에서 ODE를 풀 때, 어떤 수치적 방법을 쓰는지가 **생성 속도와 품질**에 직접 영향을 미친다.

비유로 이해하자. 산 정상에서 계곡까지 내려가는데:

- **Euler**: "현재 발밑의 경사면만 보고 한 발짝 내딛는다" → 경사가 변하면 빗나감

- **Heun (중점법)**: "한 발짝 내딛어 본 뒤, 출발점과 도착점 경사의 평균으로 다시 걷는다" → 더 정확
- **RK4**: "4개 지점의 경사를 조사한 뒤 가중 평균으로 걷는다" → 매우 정확

솔버	스텝당 모델 호출	로컬 오차	총 스텝 (동일 품질)	총 모델 호출
Euler	1회	$O(h^2)$	50+	50+
Heun	2회	$O(h^3)$	20~30	40~60
RK4	4회	$O(h^5)$	10~15	40~60

실용적 트레이드오프: Heun은 스텝당 모델을 **2번** 호출하지만, 필요한 총 스텝 수가 절반 이하로 줄어든다. 총 모델 호출 횟수가 비슷하더라도, Heun이 **더 높은 품질**을 달성한다. 실전에서는 Heun 20~30스텝이 가장 많이 사용된다.

```
# Heun 솔버 (2차 정확도)
def heun_step(model, x_t, t, t_next):
    v1 = model(x_t, t)                          # 현재 위치에서 속도
    x_next_euler = x_t + (t_next - t) * v1       # 오일러로 한 스텝
    v2 = model(x_next_euler, t_next)             # 도착점에서 속도 예
    x_next = x_t + (t_next - t) * (v1 + v2) / 2  # 두 속도의 평균으로
    return x_next
```

보충 11.4 adaLN-Zero — 왜 "Zero"가 중요한가?

DiT에서 조건 정보를 주입하는 방법으로 adaLN이 사용된다고 했다. **adaLN-Zero**는 여기에 중요한 트릭을 하나 추가한다.

표준 adaLN:

$$\text{adaLN}(h, c) = \gamma(c) \cdot \text{LayerNorm}(h) + \beta(c)$$

조건 벡터 c에서 스케일 γ와 바이어스 β를 생성하여 정규화된 은닉 상태를 조절한다.

adaLN-Zero: 각 DiT 블록의 출력에 **추가적인 스케일 파라미터 α**를 도입하고, 이를 **0으로 초기화**한다.

$$\text{output} = x + \alpha(c) \cdot \text{DiTBlock}(\text{adaLN}(x, c))$$

- α가 0에서 시작 → $\text{output} = x + 0 = x$ → **초기에는 항등 함수**
- 학습이 진행되면서 α가 점차 커짐 → 블록이 서서히 기여하기 시작

왜 이것이 중요한가? ResNet의 zero initialization과 같은 원리다:

1. 초기에 깊은 네트워크가 **얕은 네트워크처럼 동작** → 학습 안정성 확보
2. 각 블록이 점진적으로 기여를 시작 → gradient가 폭발하거나 소실되지 않음
3. 실험적으로 adaLN-Zero가 표준 adaLN보다 **FID가 유의미하게 향상**됨

```python
# adaLN-Zero 블록 (핵심)
class AdaLNZero(nn.Module):
    def __init__(self, dim):
        # 조건 벡터 c에서 6개 파라미터 생성 (γ1,β1,α1,γ2,β2,α2)
        self.adaLN_modulation = nn.Linear(dim, 6 * dim)
        nn.init.zeros_(self.adaLN_modulation.weight)  # 0으로 초기화

    def forward(self, x, c):
        gamma1, beta1, alpha1, gamma2, beta2, alpha2 = self.adaLN_mo
        # Attention 브랜치
        h = layer_norm(x) * (1 + gamma1) + beta1  # 조건부 정규화
        h = self_attention(h)
        x = x + alpha1 * h  # alpha1 초기값=0 → 처음엔 스킵!
        # FFN 브랜치
        h = layer_norm(x) * (1 + gamma2) + beta2
        h = ffn(h)
        x = x + alpha2 * h  # alpha2 초기값=0 → 처음엔 스킵!
        return x
```

보충 11.5 Consistency Distillation vs Training — 두 가지 학습 방법

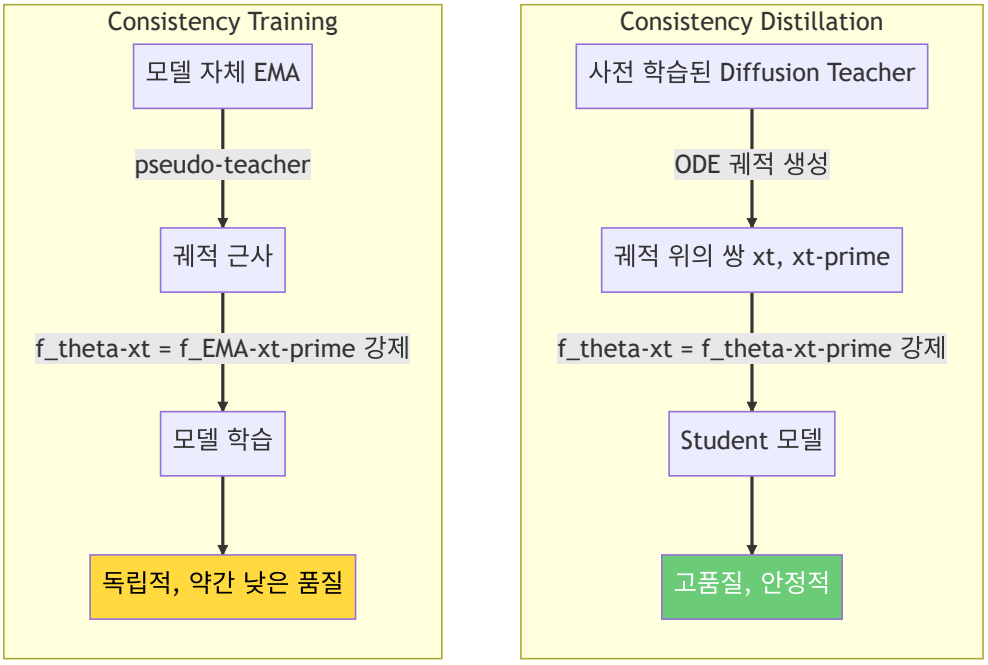
Consistency Model을 학습하는 방법은 두 가지가 있다. 각각 장단점이 뚜렷하다.

Consistency Distillation (증류):

- **전제 조건**: 사전 학습된 Diffusion 모델(teacher)이 필요
- **과정**: Teacher 모델로 ODE 궤적을 생성하고, 궤적 위의 인접한 두 점 $(x_t, x_{t'})$을 Student가 같은 x_0으로 매핑하도록 학습
- **장점**: 안정적이고 고품질
- **단점**: 비싼 사전 학습 모델이 전제

Consistency Training (독립 학습):

- **전제 조건**: 없음 (처음부터 학습)
- **과정**: 모델 자체의 EMA(Exponential Moving Average)를 pseudo-teacher로 사용
- **장점**: 독립적, 별도 teacher 불필요
- **단점**: 학습이 더 어렵고, 품질이 약간 낮음



실용적 선택: 이미 좋은 Diffusion 모델이 있다면 Distillation이 유리하다. 처음부터 빠른 생성 모델을 만들고 싶다면 Consistency Training을 쓴다.

보충 11.6 MAR & FlowAR — VQ 없이도 자기회귀가 가능하다

기존의 이미지 자기회귀 모델(VQGAN + Transformer)은 이미지를 **이산 토큰**으로 변환해야 했다. 이 양자화(VQ) 과정에서 정보 손실이 불가피하다. MAR과 FlowAR은 이 한계를 넘는 새로운 접근이다.

MAR (Masked Autoregressive Model):

기존 자기회귀가 "왼쪽→오른쪽"으로 순차 예측한다면, MAR은 BERT처럼 **랜덤 마스킹 후 예측**한다.

- 이미지 토큰 중 랜덤하게 일부를 마스킹
- 마스킹된 토큰을 예측 — 이때 cross-entropy 대신 **Diffusion 손실**을 사용
- 순차 의존성이 없어 **학습 시 병렬화 가능**
- 생성 시에는 마스크 비율을 점차 줄여가며 반복 생성

FlowAR (Flow Autoregressive Model):

자기회귀 구조와 Flow Matching을 결합한다.

- **coarse-to-fine 순서**로 자기회귀 (VAR과 유사): 저해상도 → 고해상도
- 각 스케일의 토큰 생성에 **Flow Matching**을 사용 (이산이 아닌 연속 생성)
- AR이 전체 구조를 제공하고, FM이 각 스케일의 연속적 디테일을 생성

두 접근의 공통된 혁신: VQ 없이 **연속 공간에서 직접 자기회귀 생성**이 가능하다. 양자화 병목이 사라지면서 더 높은 품질이 달성된다. 또한 Transformer의 **Scaling Law**도 그대

로 적용할 수 있다.

보충 12.5 DreamFusion & SDS — 2D 지식으로 3D를 만든다

문제: 3D 생성 모델을 학습하려면 3D 데이터가 필요한데, 고품질 3D 데이터는 극히 부족하다.

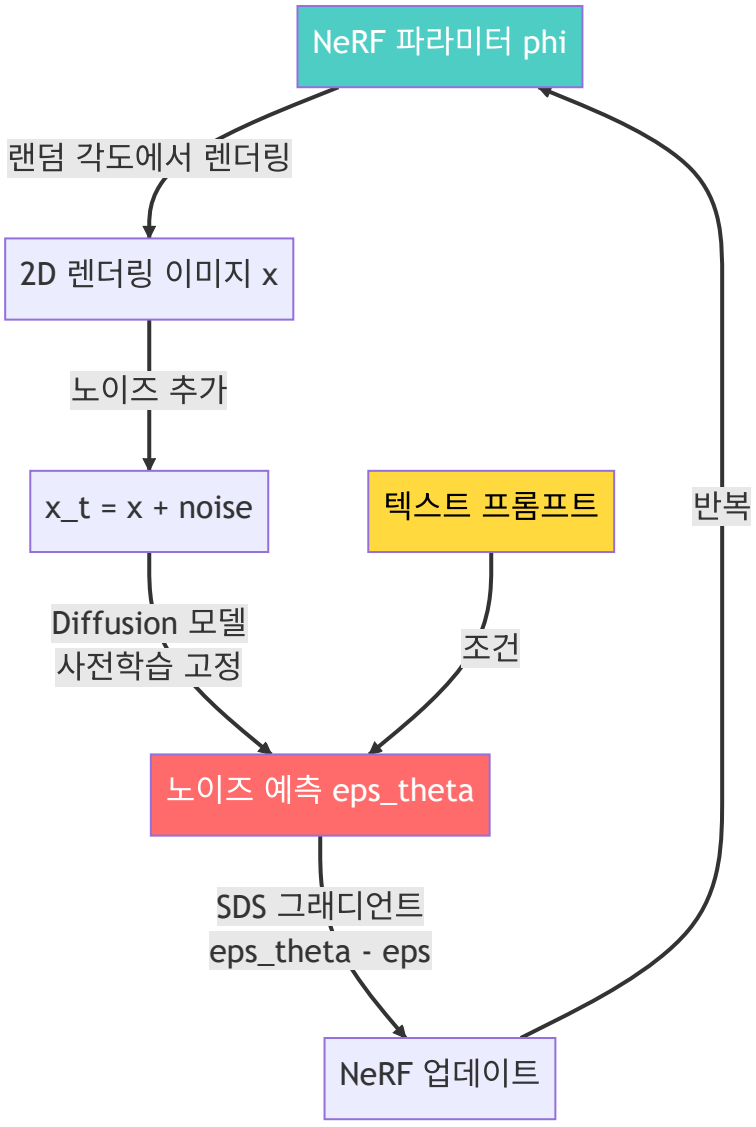
DreamFusion의 아이디어: 3D 데이터 없이, 이미 학습된 2D Diffusion 모델의 "시각적 지식"을 활용하여 3D 모델(NeRF)을 최적화한다.

Score Distillation Sampling (SDS) 과정:

1. NeRF를 **랜덤 각도**에서 렌더링 → 2D 이미지 x 생성
2. 렌더링된 이미지에 노이즈 추가 → x_t
3. 사전 학습된 Diffusion 모델이 노이즈 예측 → $\epsilon_\theta(x_t, t, \text{text})$
4. SDS 그래디언트 계산:

$$\nabla_\phi \mathcal{L}_{\text{SDS}} = \mathbb{E}_{t,\epsilon}\left[(\epsilon_\theta(x_t, t, y) - \epsilon) \cdot \frac{\partial x}{\partial \phi}\right]$$

5. 이 그래디언트로 NeRF 파라미터 ϕ를 업데이트



직관: Diffusion 모델에게 이렇게 묻는 것이다: "이 렌더링 이미지가 '{text}'처럼 보이는가? 아니라면, 어떻게 고쳐야 하는가?" 그 피드백을 NeRF에 전달하여 3D 형상을 수정한다.

비유: 조각가가 점토로 형상을 만드는데, 다양한 각도에서 사진을 찍어 미술 평론가(Diffusion 모델)에게 보여준다. 평론가가 "이 각도에서 보면 고양이 같지 않다"고 피드백하면, 조각가가 그에 맞게 형상을 수정한다.

보충 12.6 Multi-View Diffusion — 여러 각도를 동시에 생성

SDS의 문제: NeRF를 반복 최적화하므로 느리고 (수 시간), **Janus 문제** — 모든 각도에서 "앞면"이 보이는 아티팩트 — 가 발생한다.

Janus 문제의 원인: SDS는 각 각도를 **독립적으로** 평가한다. "앞에서 보면 고양이 얼굴"이어야 하고 "뒤에서 보면 고양이 얼굴"이어야 하면 → 모든 면에 얼굴이 생긴다.

Multi-View Diffusion의 해결책: 여러 시점의 이미지를 **동시에** 생성하여, 시점 간 **일관성을 보장**한다.

접근 방법:

1. 멀티뷰 데이터셋(Objaverse 등)에서 Diffusion 모델을 학습
2. 디노이징 시 여러 시점의 정보를 Cross-Attention으로 공유
3. "앞면"을 생성할 때 "뒷면"의 정보를 참조 → 일관된 3D 구조

대표 모델:

- **Zero-1-to-3**: 단일 이미지 → 새로운 시점 이미지 생성
- **MVDream**: 텍스트 → 4개 시점 동시 생성
- **Wonder3D**: 단일 이미지 → 6개 시점 + 법선 맵(normal map) 동시 생성

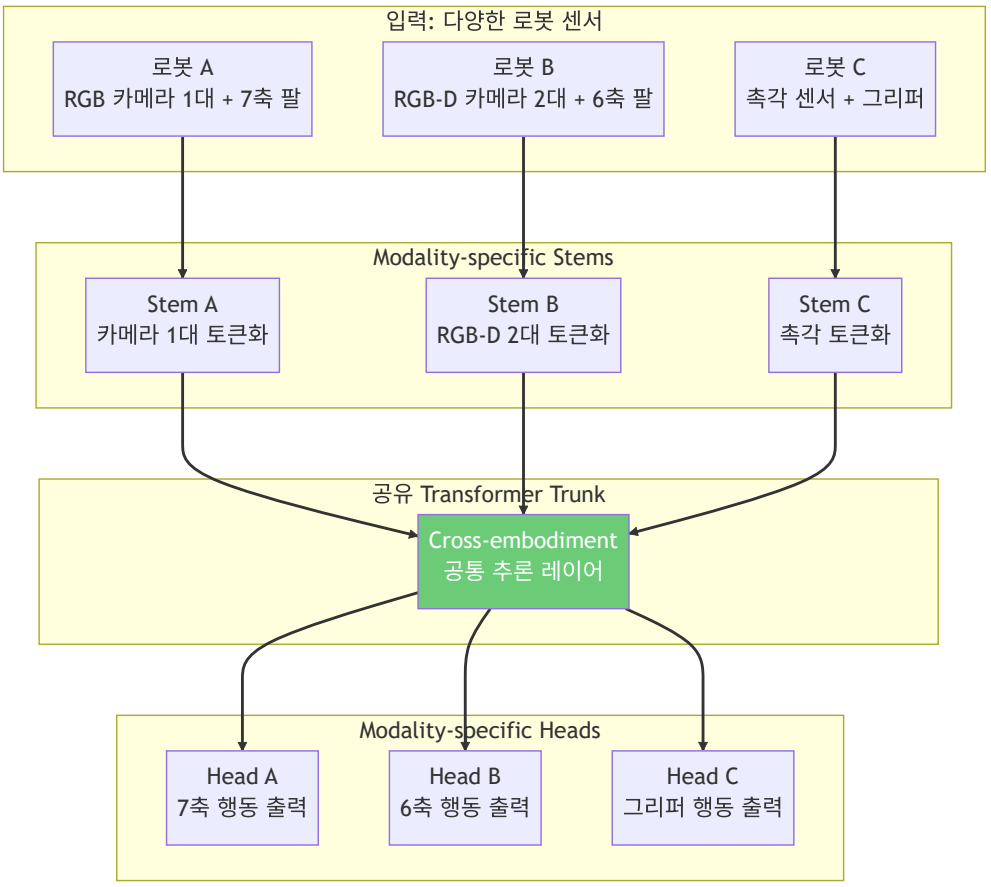
이 접근은 SDS의 느린 최적화를 피하고, **피드포워드(한 번에)**로 3D 정보를 추출할 수 있어 수 초 만에 3D 생성이 가능하다.

보충 12.7 HPT — 이종 로봇 데이터를 하나의 모델로

문제: 로봇마다 센서와 액추에이터가 다르다. 카메라가 1개인 로봇, 2개인 로봇, 촉각 센서가 있는 로봇, 7축 팔, 6축 팔... 데이터 형식이 제각각이라 하나의 모델로 학습하기 어렵다.

HPT (Heterogeneous Pre-trained Transformer)의 아이디어: 언어 모델이 다양한 언어를 하나의 모델로 처리하듯, **다양한 로봇의 데이터를 하나의 Transformer로 처리**한다.

HPT 아키텍처:



세 가지 구성 요소:

1. **Modality-specific Stems (입력 변환기)**: 각 로봇의 센서 데이터를 **공통 토큰 형식**으로 변환. 카메라 이미지는 ViT로, 관절 각도는 MLP로 토큰화.
2. **공유 Transformer Trunk (공통 추론기)**: 모든 로봇에서 공유. "물체를 잡으려면 물체 위치로 이동해야 한다"는 보편적 지식을 여기서 학습.
3. **Modality-specific Heads (출력 변환기)**: 공통 표현을 각 로봇의 **행동 형식으로 변환**. 7축 팔은 7차원 출력, 6축 팔은 6차원 출력.

비유: "동시통역사"와 같다. 다양한 언어(로봇 형식)로 된 입력을 공통 의미(공유 Trunk)로 이해하고, 다시 적절한 언어(행동 형식)로 번역한다.

핵심 발견 — Cross-Embodiment Transfer: 팔 로봇에서 배운 "물체 잡기" 지식이 다리 로봇의 "물체 밟지 않기"에 도움이 된다. 서로 다른 형태의 로봇 사이에서도 **조작(manipulation)에 관한 보편적 지식이 전이**된다.

학습 방식: 다양한 로봇 데이터셋으로 사전 학습(pre-train) → 특정 로봇에 미세 조정(fine-tune). 이는 LLM의 "대규모 사전학습 → 다운스트림 미세조정" 패러다임과 정확히 동일하다.